Colibri auf dem Heim-PC — echter Test: 35B und 744B ohne GPU

Die Behauptung aus dem Reel: „744 Milliarden Parameter auf einem normalen Computer“ — hier steht, was wir selbst installiert und gemessen haben, nicht was auf der Projektseite steht.

Kiste: ki · Ubuntu 26.04.1 LTS | Intel Core i5-10500T (6C/12T, 2.3-3.8 GHz) | 30 GB RAM · SSD · keine GPU

Urteil: Behauptung stimmt

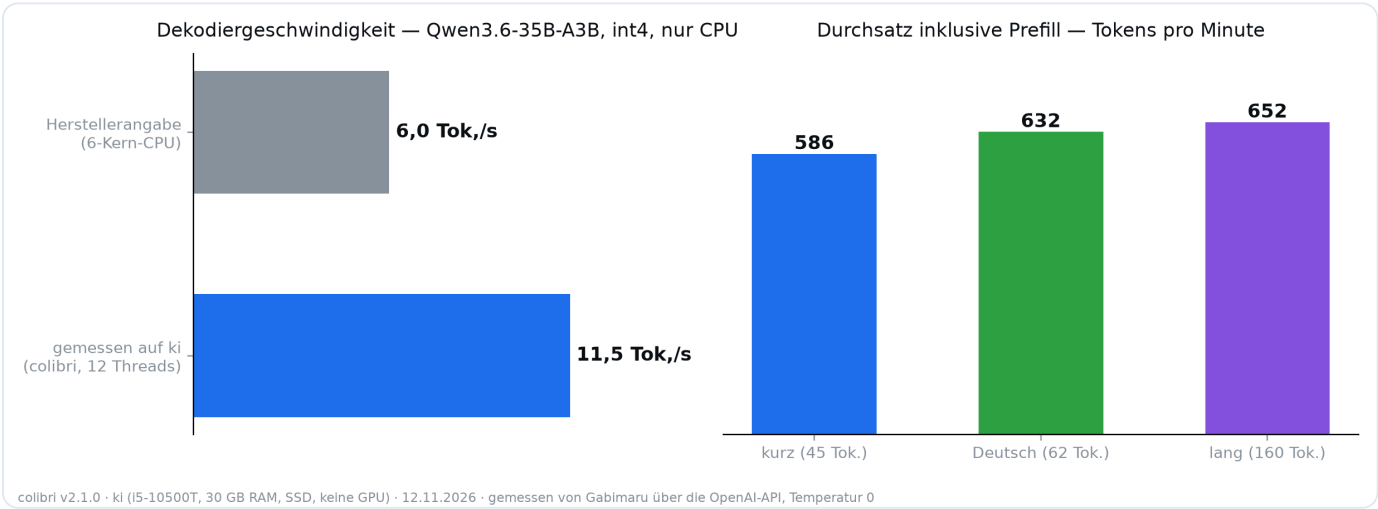
Die Engine **colibri v2.1.0** (Apache-2.0, in C geschrieben) hält den dichten Teil des Modells im RAM und streamt die gerouteten Experten von der SSD. Auf dieser Kiste aus dem offiziellen Repo gebaut (gcc 15.2), Modell Qwen3.6-35B-A3B im int4-Container (23 GB) über die OpenAI-kompatible API bedient. Alle Zahlen unten stammen aus drei echten Anfragen mit `temperature 0`.

DEKODIERGESCHWINDIGKEIT	DURCHSATZ	ERSTER TOKEN	SPEICHER
11,5	**623**	**0,53 s**	**14,6**
Tokens pro Sekunde	Tokens pro Minute (inkl. Prefill)	Time to first token	GB RAM im Betrieb

Messung auf ki — echte API-Aufrufe, drei Prompts

Prompt	Tokens rein	Tokens raus	TTFT (s)	Tok./s	Tokens/Min.	SSD gelesen (MB)
`short_en`	31	45	0,72	**11,56**	586	54
`german`	32	62	0,53	**11,57**	632	54
`long_en`	34	160	0,77	**11,46**	652	156

Das Modell liegt vollständig im RAM, deshalb fast kein Plattenlesen und deshalb schneller als die Herstellerangabe von 6 Tokens/s auf einer 6-Kern-CPU: **11,5 Tok./s** statt 6,0 — 12 Threads und ein 30-GB-Cache für Experten.



Links: gemessene Geschwindigkeit gegen die Herstellerangabe. Rechts: Tokens pro Minute für die drei Prompts.



Live-Dashboard desselben Servers

Links: die Chat-Oberfläche von colibri auf genau dieser Kiste, verbunden mit Qwen3.6-35B-A3B; die deutsche Antwort kam in wenigen Sekunden. Dieselbe Engine liefert eine OpenAI-kompatible API auf Port 8000 und eine Expertenkarte, die zeigt, welche Experten pro Token von der Platte gelesen werden.

Installation unter Ubuntu (genau das wurde gemacht)

```
# 1) prerequisites - Ubuntu 24.04 / 26.04
sudo apt install -y git python3 build-essential
git clone --depth 1 https://github.com/JustVugg/colibri ~/colibri-moe/colibri
cd ~/colibri-moe/colibri

# 2) build the CPU engine (no GPU) - a few seconds
make -C c portable -j$(nproc)

# 3) which model fits this machine? (no download)
python3 c/coli setup --list

# 4) install a model + serve the OpenAI-compatible API
python3 c/coli setup --model qwen36-35b --yes
curl -s http://127.0.0.1:8000/v1/models
```

Was auf ki funktioniert hat: Build mit gcc 15.2 ohne Fehler; das 23-GB-Modell war in rund 10 Minuten geladen (37,7 MB/s), die API stand auf Port 8000.

Stolperfalle: vor dem Download immer `setup --list` laufen lassen. Der Engpass ist der RAM, nicht die Platte: der dichte Teil muss in den RAM passen, sonst startet die Engine nicht.

Welches Modell passt auf diese Kiste? (Ausgabe des Werkzeugs)

Modell	Download	Status	Begründung
`qwen36-35b`	23 GB	**good**	passt komplett in den RAM (20 GB)
`qwen3-coder-30b`	19 GB	**good**	passt komplett in den RAM (18 GB)
`deepseek-v4-flash-reap`	85 GB	**good**	dense 7,9 GB im RAM, Experten vom SSD
`deepseek-v4-flash`	167 GB	**good**	dense 7,9 GB im RAM
`mimo-v2.6-flash`	172 GB	**tight**	zu wenig RAM (32 von 52 GB)
`qwen38-flash-next`	186 GB	**good**	dense 8 GB im RAM
`glm-5.3`	419 GB	**good**	dense 9,9 GB im RAM

Geprüft wurden 11 Modellfamilien. `glm-5.2` mit 744 Milliarden Parametern ist auf dieser Kiste „good“, weil nur 9,9 GB im RAM bleiben müssen. `kimi-k3` (1,5 Billionen) braucht 1594 GB Platte und passt bei 753 GB frei nicht.

Und die 744 Milliarden Parameter?

GLM-5.2 · 429 GB · int4

Der Download läuft; bei gemessenen 38,5 MB/s sind es noch etwa **3,0 Stunden**. Geschwindigkeit und Tokenverbrauch dieses Modells werden automatisch gemessen und in genau diesen Bericht nachgetragen.

3,0 %

Wo das gespeichert ist

Was	Pfad / Link
Skill	`skills/devops/colibri-local-moe`
Benchmark-Skript	`workspace/colibri-moe/bench_api.py`
Rohdaten + dieses PDF	`github.com/ahmadvahidi/toolbox · docs/reports/`